

Trust by Construction: Secure-by-Design Patterns for Agentic AI in Learning Institutions

Scale Intelligence Without Scaling Implicit Authority

Rachana Srivastava

Enterprise Architect | AI Systems Researcher

UNU Macau AI Conference 2026 | Extended abstract

AI × Education: AI for Learning, Learning for AI

1. Authority before autonomy

In a learning institution, AI agents now read student records, change school systems and delegate to other agents. A transcript assistant should retrieve the named record, propose a correction and deliver the approved result to the registrar. It should not open another student's file because a document tells it to, approve its own correction, or keep releasing data after revocation. Indirect prompt injection can redirect tool-integrated agents [1].

The harder problem is that permitted steps combine into an outcome no one allowed. A reader, a summariser and a publisher may each hold a sound tool list and still form a path from a student transcript to a public page. No one agent misbehaves. The chain does.

Trust by Construction treats every agent, its coordinator included, as an untrusted proposer. Separate services mediate protected reads, delegation, effects and disclosure under one rule: proof before power. The contribution is ten design patterns that turn policy into executable refusals, a composition contract that holds when agents spawn agents, and an offline evidence kit a school can rerun on a laptop.

The work builds on control-flow separation [2], data-flow control [3] and agent design patterns [4], which bound what one agent may do; the open question is what permitted agents do together. The aim is an integration and test method, not a new primitive. Proof means checking stated rules against trusted state, not that a model is truthful or a grade fair.

2. Ten patterns, one boundary, and a swarm contract

The model makes proposals. Separate, trusted software decides which operations may run. Scoped leases let an agent call the gateway without letting it rewrite the gateway's policy. The ten patterns are a declared authority ceiling (P1), a task contract (P2), a credentialless runtime (P3), attenuated delegation (P4), mediated context (P5), reauthorised memory (P6), typed effects (P7), sealed release (P8), a restricting monitor (P9) and a declared task graph (P10).

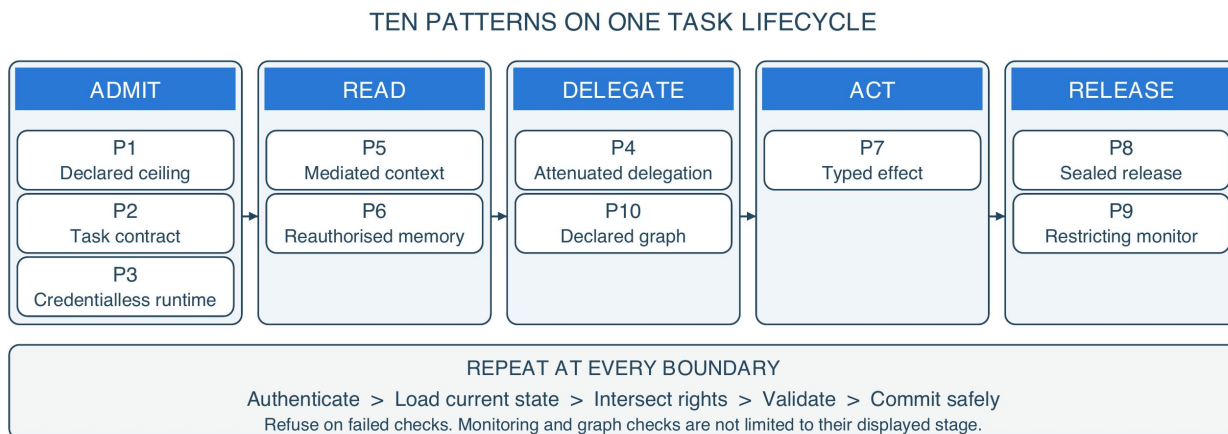


Figure 1. Ten patterns across one task lifecycle. Columns show each pattern's primary stage.

The threat model allows malicious or colluding agents, poisoned content and a monitor that is wrong. It assumes trusted enforcement code, protected state, known operators and adapters that cannot be bypassed. A task purpose picks an approved operation and resource policy; it does not establish what the agent means.

Every boundary does the same five things: authenticate the caller, load current state, intersect rights, validate the exact operation, then commit. The case makes this concrete. A registrar asks for a grade correction. The assistant may read the named record, because the lease names that record and no other. It drafts a change, but the write is refused until the instructor confirms, because the contract names a second approver. The receipt binds that approval to that exact draft, so a later, different write cannot reuse it. Revoke the grant mid-task and the queued write fails, even though it was approved when it was drafted.

ONE GRADE CORRECTION: WHAT THE ASSISTANT ASKS, WHAT THE GATE DECIDES		
STEP	THE ASSISTANT PROPOSES (UNTRUSTED)	THE ENFORCEMENT GATE DECIDES (TRUSTED)
LEASE	Work on the grade for record S-1042	→ Lease issued naming that one record. Any other file is out of scope. SCOPED
READ	Read S-1042	→ Within the lease. The read is allowed and recorded. ALLOWED
DRAFT	Write the corrected grade	→ Refused. The contract names a second approver. REFUSED
APPROVE	Instructor confirms this draft	→ The receipt binds that approval to this draft digest. BOUND
COMMIT	Write the approved draft	→ Epoch, scope and approval revalidated, then committed. ALLOWED
SUBSTITUTE	Write a different draft under that approval	→ Refused. The receipt names another digest. REFUSED
REVOKE	Queued write, after the grant is withdrawn	→ Refused. A stale epoch fails revalidation. REFUSED

No step depends on the assistant behaving. Every verdict is a check of stated rules against trusted state.
The same seven steps run for a medical record or a benefits claim; only the contract and the lease change.

Figure 2. One grade correction, end to end. Every step is a check of stated rules against trusted state, so no step depends on the assistant behaving.

The design hypothesis is falsifiable: under an unchanged contract, adding workers must not admit a prohibited effect, information path or total spend. That must be checked on the whole composition, not on each worker alone.

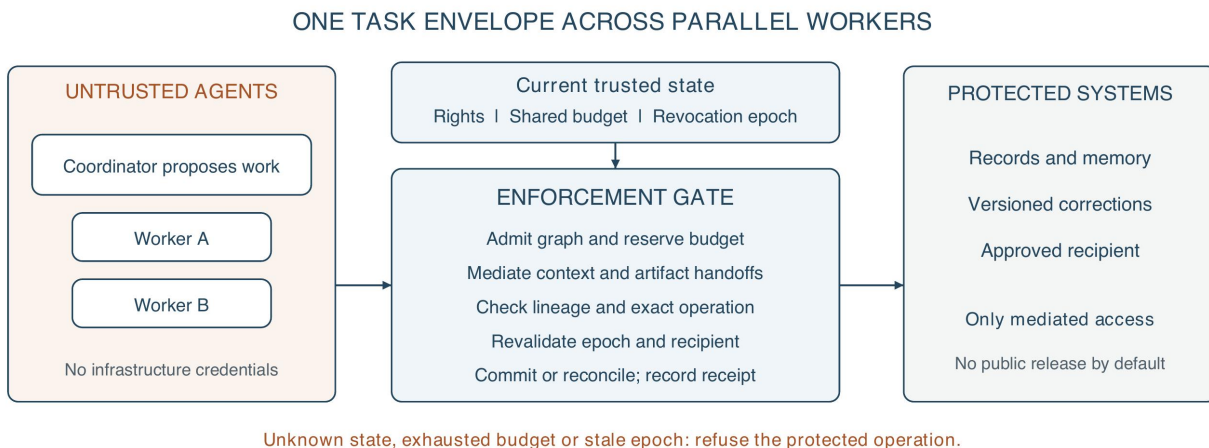


Figure 3. Proposed swarm extension. Parallel workers share one task budget and revocation epoch; every handoff crosses enforcement. The diagram specifies architecture, not measured distributed performance.

A swarm is a set of agents coordinated on one institutional task. The coordinator may split work and propose workers, but spawning is itself mediated. Each child gets its own identity, an expiry and authority no wider than its ancestors, task and workload ceiling; per-agent identity is what makes a swarm reviewable at all [7].

Replanning changes a proposed task graph, and a trusted gate must admit each new node and edge before use. Agreement among agents is evidence for review, never a credential.

Parallelism must not multiply resources. All descendants draw on task-wide limits for calls, records read, bytes released and consequential effects, plus a workload ceiling across tasks. Depth, population and time limits bound recursive spawning. Atomic reservations precede dispatch, and completion or cancellation settles them without double spending. External effects need adapter reconciliation when the outcome is unclear: a database rollback cannot unsend an email.

Information composes the same way. Trusted mediation binds source labels to artifacts and messages, so agents cannot strip them by rewording or dropping citations. Outputs inherit restrictions from all exposed context, including retained memory. Reuse with a clean context needs a new isolated context; an agent's claim to have forgotten is not enough. Combining inputs intersects permitted recipients and purposes, and any relaxation needs a separately authorised release. This can over-restrict useful summaries, a deliberate trade.



Figure 4. Composition changes the security question. A tool allowlist does not stop a permitted publisher disclosing another agent's protected input. Labels and recipient checks must survive the handoff; agreement among agents supplies no release authority.

At fan-in, a trusted gate checks provenance, current authority, artifact identity and destination before accepting results. Revocation raises the task epoch, so queued work, stale leases and later release chunks fail revalidation. Disclosed information cannot be recalled. A fresh check alone leaves a check-then-act race: strict revocation needs authorisation and commit serialised against revocation, or an adapter that rejects stale epochs. Partitioned workers must stop when freshness cannot be shown, trading availability for bounded authority. Physical isolation, egress control and trustworthy connectors stay deployment duties.

3. Evidence, falsification and impact

The FSSAI-RA reference kernel runs offline, without model weights or a GPU. Its recorded six-profile evaluation explores 55,440 bounded configurations with no violation, 180 hostile scenarios, all contained, and 58 benign workflows, all completed [5]. These profiles reuse an authored scenario suite: the counts and that state space are linked synthetic checks, not 180 independent attacks, outside validation or reliability estimates.

Three findings do more work than the totals. First, composition is not covered by local correctness. Against ten delegation attacks, an unguarded chain contained none, validating each hop against its immediate delegator contained two, and verifying the whole chain contained ten. The middle arm is a real control and what a careful engineer builds. It stops the two failures a single hop can see, a widened scope and an untrusted key, while confused deputies, bearer-chain reuse, unrooted chains and depth evasion walk through. Each of the nine controls is load-bearing: removing one restores its harm.

WHERE LOCAL CORRECTNESS FAILS

Ten delegation risk classes against three architectures, from a live run of the offline kernel

RISK CLASS	A UNGUARDED	B PER-HOP CHECK	C WHOLE CHAIN	WHY ARM C REFUSES
Bearer chain reuse	○	○	●	Requester Not Chain Leaf
Confused deputy	○	○	●	Out Of Effective Scope
Undisclosed beneficiary	○	○	●	Beneficiary Scope Exceeded
Scope reamplification	○	●	●	Scope Not Attenuated
Authority laundering	○	○	●	Chain Cycle
Orphaned delegation	○	○	●	Delegation Outlives Delegator
Unrooted chain	○	○	●	Chain Not Rooted
Depth evasion	○	○	●	Depth Exceeded
Machine delegated consequence	○	○	●	Consequence Not Delegable
Forged hop	○	●	●	Delegation Key Untrusted
Benign two hop (utility control)	●	●	●	Admitted
HOSTILE CHAINS CONTAINED	0 / 10	2 / 10	10 / 10	

● contained
○ passed through

Arm B is a correct implementation of the realistic control, not a strawman. Removing any one of the 9 named controls restores its harm.

Figure 5. Where local correctness fails. Ten delegation risk classes against three architectures, from a live run of the offline kernel; the benign chain completes in all three.

Second, sequence testing found what step testing missed. A stateful harness caught a summary drafted while access was valid but released after consent was withdrawn, which single-step enumeration passed. Release now rechecks live state; removing that control makes the violation reappear.

Third, a second domain found what one could not. The academic-records profile exposed a declared approval role the runtime silently ignored, because the enforcement map was built only from consequential rules.

Eight claims are implemented and bound to a named offline test: delegation cannot enlarge authority, parallel siblings cannot multiply a budget, separate processes cannot race a commit, composition cannot erase a restriction, memory cannot carry a restriction away, revocation invalidates in-flight authority, a release cannot resume after revocation, and checking each step is not checking the sequence. The same bound across distributed workers is specified, not yet evaluated. Undoing a delivered external effect is not claimed.

WHAT IS BUILT AND WHAT IS PROPOSED

CLAIM	MECHANISM THAT ENFORCES IT	STATUS	EVIDENCE IN THE REFERENCE KERNEL
Delegation cannot enlarge authority	Attenuated lease; ancestor intersection	BUILT	test_lease_cannot_expand_any_authority_axis
Parallel siblings cannot multiply a budget	One task envelope; atomic reservation	BUILT	test_aggregate_sibling_budget_does_not_multiply
Separate processes cannot race a commit	Serialised authorisation and commit	BUILT	test_independent_processes_serialize_budget_and_release
Composition cannot erase a restriction	Bound source labels; recipient checks	BUILT	test_potential_source_to_sink_path_denied_even_for_clean_message
Memory cannot carry a restriction away	Reauthorised memory; restart persistence	BUILT	test_restart_preserves_memory_restrictions_budget_and_rejects_legacy_adapter
Revocation invalidates in-flight authority	Task epoch revalidated at every use	BUILT	test_effect_approval_substitution_and_guardian_epoch_invalidation
A release cannot resume after revocation	Per-chunk reauthorisation	BUILT	test_stream_reauthorizes_every_chunk_and_bulk_cannot_bypass
Checking each step is not checking the sequence	Stateful harness against a reference model	BUILT	test_stateful_sequences_hold_and_find_counterexamples_when_a_control_is_removed
The same bound holds across distributed workers	Shared epoch and reservations	PROPOSED	Evaluation plan, section 4
A delivered external effect can be undone	Adapter reconciliation only	NOT CLAIMED	Stated limit

Every BUILT row names an offline regression test. PROPOSED rows are obligations this abstract specifies and does not evaluate.

Figure 6. What is built and what is proposed. Each claim names the mechanism that enforces it, its status and the offline test that exercises it.

The next evaluation should compare an unmediated swarm, separate local controls, shared task enforcement and a deny-all baseline under identical workloads, injecting malicious coordination, repeated spawning, label stripping, duplicate delivery, revocation during execution and partitions. It should report prohibited effects beside benign completion, false refusals, tail latency and cost, and raise worker count and depth to test non-amplification. It should also run against a public injection benchmark [8]. A single prohibited composite outcome falsifies the claim.

Security remains conditional on complete mediation and sound policy. A compromised enforcer, covert channel or permissive policy can invalidate containment, and a malicious monitor can deny service without escalating privilege. Authorised actions may still be inaccurate, discriminatory or educationally harmful, so review, appeal, accessibility and pedagogical evaluation remain necessary. A decision receipt binds policy version, task epoch, source lineage, artifact digest, recipient and outcome, so review need not treat model reasoning as proof. Receipts need access controls and retention limits, so the audit trail is not another store of student data.

For institutions, this makes trust something a buyer can demand and an auditor can rerun: named enforcement points, a filled task contract, declared review limits and fresh evidence. Procurement can require the refusal suite before signing; a regulator can check purpose and revocation as running controls, not policy text. Because the contract sits outside the model, a school can change model or vendor and rerun the same tests, so the safety case is not rebuilt each time. Sovereignty becomes control of keys, data flow and exit, not server location.

4. Conclusion: scale intelligence, bound authority

For AI for Learning, a registrar can inspect record access, correction authority and release destinations. Institutions should deploy bounded, reversible workflows first, qualify connectors, and evaluate with staff and students before expanding autonomy. A named owner must handle escalation and appeals, and students must be told how to contest a result. Human review capacity is finite: workloads exceeding it should queue or reduce autonomy rather than treat a nominal approval step as effective oversight.

For Learning for AI, learners can inspect a forged approval, poisoned memory and a colluding agent chain, then observe which boundary refuses each action. This complements UNESCO's human-centred AI competency agenda [6] with practical understanding of authority and redress, and offline verification lowers the kit a teaching lab needs.

The design supports SDG 4 through accountable school workflows and SDG 16 through explicit, contestable institutional authority. Because the contract is enforced outside the model, institutions can require the same authority contract and refusal suite across model and vendor changes, so governance attaches to a workflow whose authority can be withdrawn rather than to a vendor's assurances.

Scaling intelligence is not the risk; scaling implicit authority is. Increasing autonomy should expand useful computation within a declared mandate. It should never silently expand the mandate itself.

References

- [1] Zhan, Q., et al. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL. <https://arxiv.org/abs/2403.02691>
- [2] Debenedetti, E., et al. (2025). Defeating Prompt Injections by Design. arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [3] Costa, M., et al. (2025). Securing AI Agents with Information-Flow Control. arXiv:2505.23643. <https://arxiv.org/abs/2505.23643>
- [4] Beurer-Kellner, L., et al. (2025). Design Patterns for Securing LLM Agents against Prompt Injections. arXiv:2506.08837. <https://arxiv.org/abs/2506.08837>
- [5] Srivastava, R. (2026). FSSAI-RA reference implementation. Repository snapshot 9bb35a1: [fssai-raftree/9bb35a1](https://github.com/genaiworks/fssai-raftree/9bb35a1)
- [6] UNESCO (2024). AI Competency Framework for Students. <https://unesdoc.unesco.org/ark:/48223/pf0000391105>
- [7] Chan, A., et al. (2024). Visibility into AI Agents. ACM FAccT. <https://arxiv.org/abs/2401.13138>
- [8] Debenedetti, E., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS Datasets and Benchmarks. <https://arxiv.org/abs/2406.13352>